Annex I
Protocol Enforcement Mechanics
BEAP™ / WR Desk™ Canonical Specification

I.0 Status, Scope, and Relationship to Canon

Status

This Annex is **normative** unless explicitly marked as informative.
It defines mandatory enforcement mechanics, architectural constraints, operational rules, and interaction boundaries for BEAP™-conformant implementations.

Scope Boundary

This Annex does **not** redefine or extend:

- Wire formats
- Capsule serialization formats
- Cryptographic primitives
- Transport mechanisms
- PoAE™ execution semantics

These elements remain exclusively defined in the Canon and its Normative Annex.

This Annex defines **how enforcement is implemented** at runtime and architectural level.

Relationship to the Canon

- The Canon defines **what** is required.
- Annex I defines **how** those requirements are enforced.
- Nothing in this Annex weakens or contradicts Canon requirements.

I.1 Architectural Separation Principles

I.1.1 Separation of Probabilistic Reasoning and Governance Enforcement

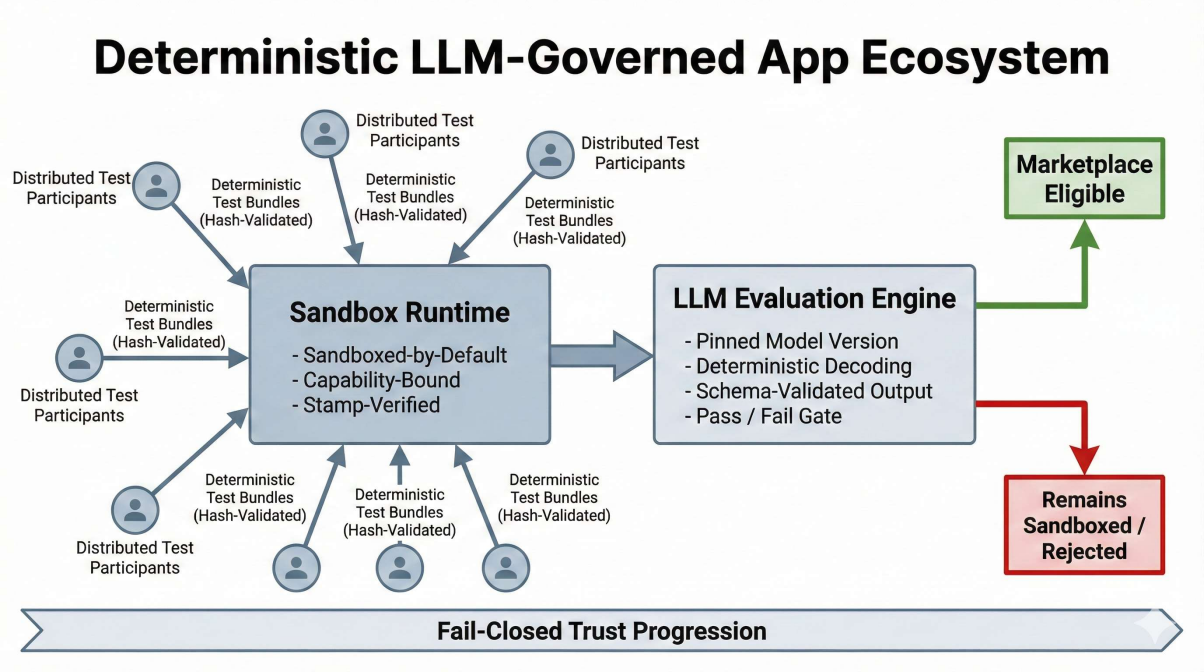
The system SHALL maintain strict architectural separation between:

- Probabilistic reasoning components (e.g., LWM / Optimando)
- Deterministic governance enforcement layers (capability engine, policy engine, PoAE™ validation, attestation enforcement)

The LWM:

- MAY propose actions
- MAY propose workflow transitions
- MAY request tool invocations
- SHALL NOT execute tools directly
- SHALL NOT elevate privileges
- SHALL NOT modify capability scopes
- SHALL NOT override policy decisions

All execution authority SHALL remain external to the model.



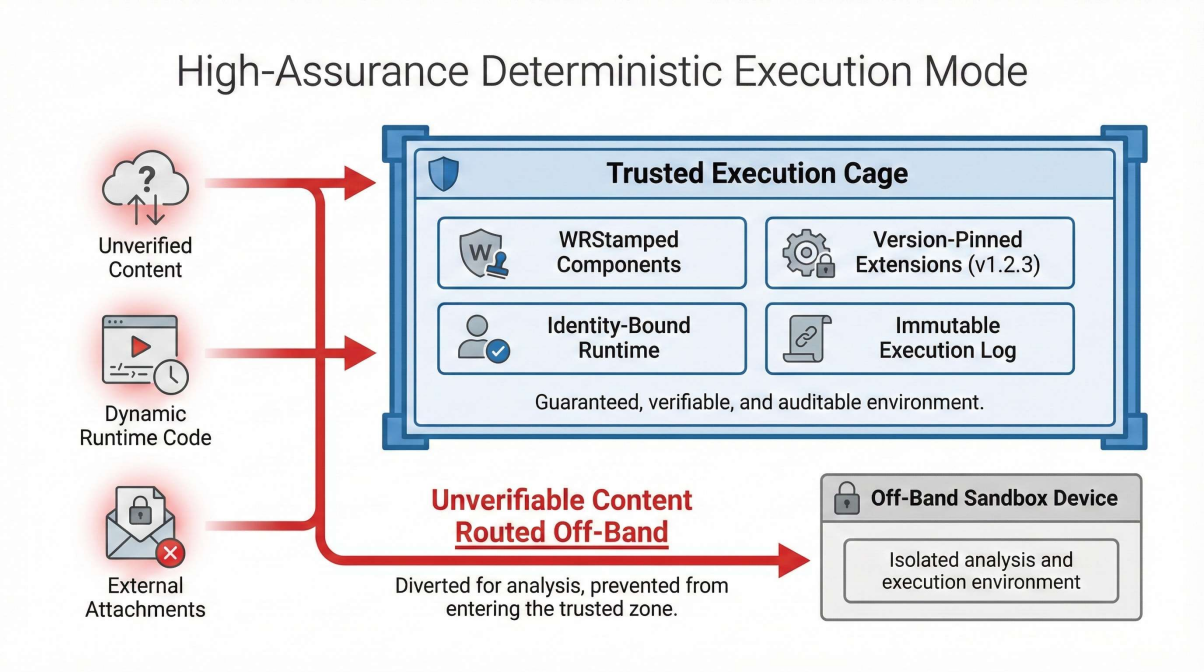
I.2 Mandatory Sub-Orchestrator Architecture

I.2.1 Primary and Sandbox Sub-Orchestrators (Normative)

Every conforming WR Desk™ implementation SHALL include:

- One Primary (Host) Orchestrator
- At least one Sandbox Sub-Orchestrator

The sandbox sub-orchestrator is a **mandatory** component and SHALL NOT be optional.



I.2.2 Mandatory Sandbox Execution Boundary

The following SHALL execute exclusively inside the sandbox sub-orchestrator:

- Sandboxed applications and extensions

- Unverified or newly ingested artifacts
- Workflow capsules prior to trust escalation
- External-origin artefacts (emails, attachments, third-party content)

The sandbox sub-orchestrator SHALL:

- Enforce declared capability boundaries
- Enforce version pinning
- Enforce stamp validation
- Execute mandatory deterministic pre-execution checks
- Operate fail-closed

I.2.3 Non-Bypassable Pre-Execution Enforcement

Before any sandboxed artifact or workflow capsule may execute in a trusted or production context:

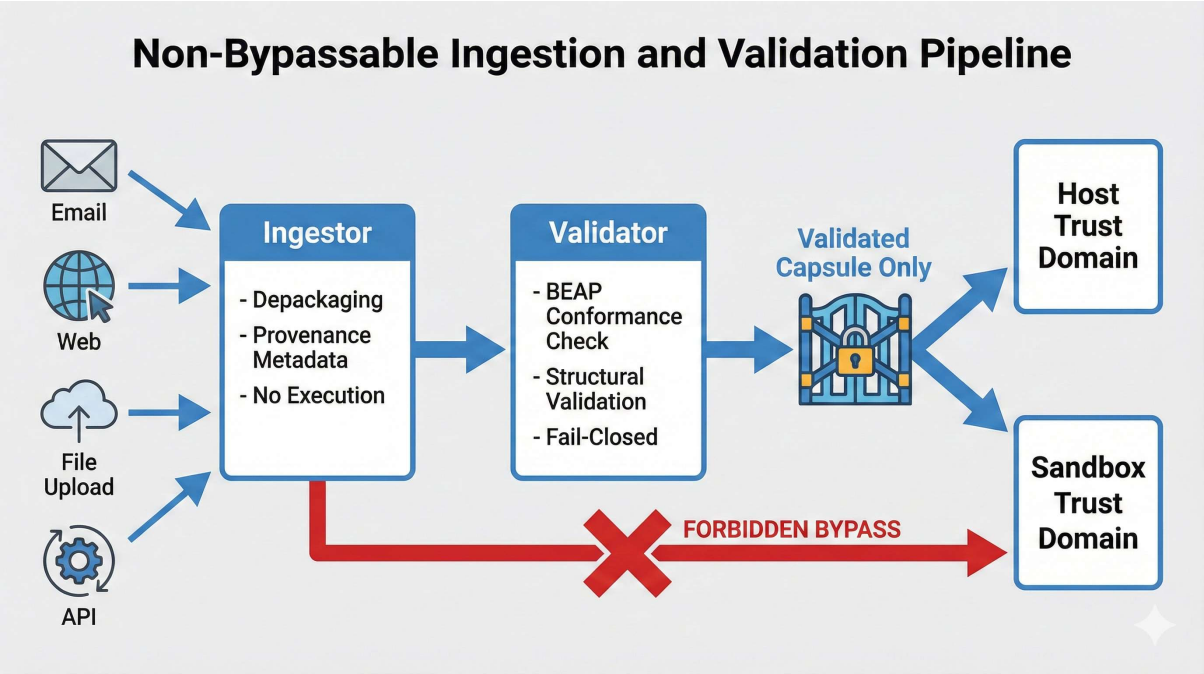
- Mandatory deterministic test routines SHALL be executed
- Stamp integrity SHALL be verified
- Capability declarations SHALL be validated
- Policy conformance SHALL be confirmed

These checks SHALL NOT be deferred, disabled, or bypassed.

Unverified logic SHALL NEVER execute in a trusted domain.

I.2.4 Physical Separation Requirement

Sandbox sub-orchestrators SHALL execute on physically separated hardware. Virtualization or containerization alone SHALL NOT be considered sufficient isolation.



I.3 Ingestion and Validation Pipeline (Normative)

I.3.1 Ingestor–Validator Separation

The system SHALL implement a mandatory two-stage ingestion pipeline:

- Ingestor
- Validator

These components SHALL be logically separated with a non-bypassable interface.

I.3.2 Ingestor Responsibilities

The Ingestor SHALL:

- Act as exclusive entry point for all external inputs
- Depackage inputs into candidate BEAP™ capsules
- Attach provenance metadata

The Ingestor SHALL NOT:

- Perform full protocol validation
- Render content
- Execute payloads
- Distribute capsules

I.3.3 Validator Responsibilities

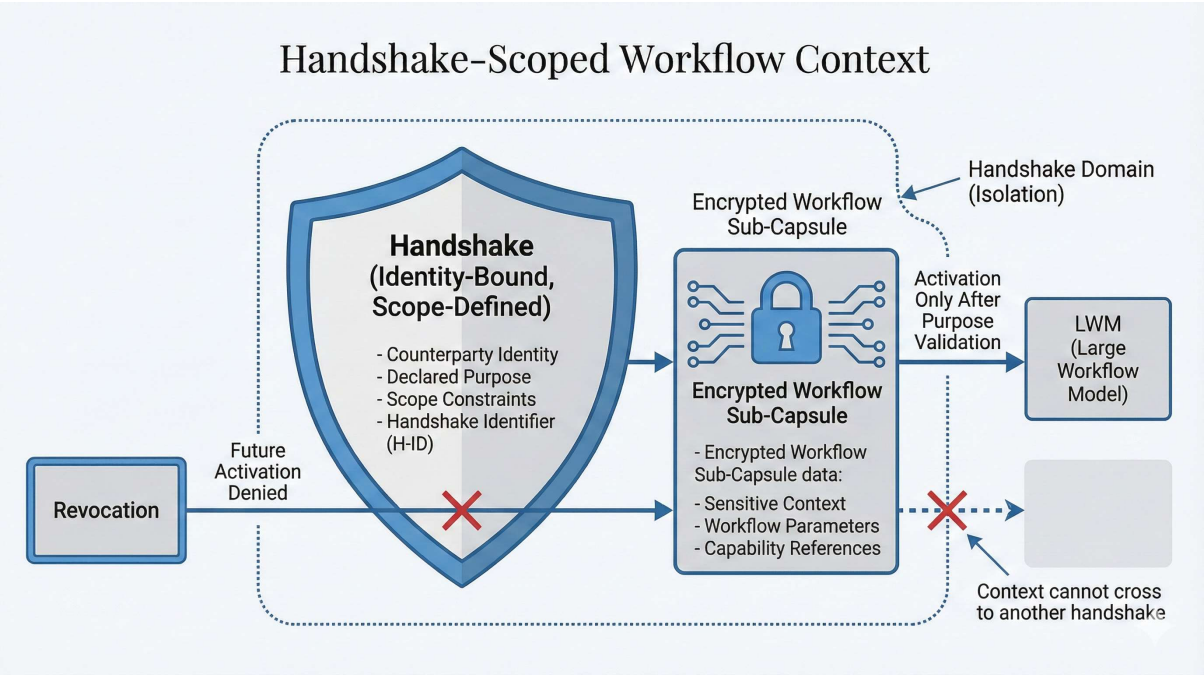
The Validator SHALL:

- Perform strict BEAP™ conformance validation
- Validate structural integrity and required metadata
- Fail closed on any error
- Mark capsules as validated only upon full verification

No component other than the Validator may mark a capsule as validated.

I.3.4 Distribution Gate

Only capsules explicitly marked as validated SHALL be distributed to any trust domain.



I.4 Workflow Capsule Governance

I.4.1 Workflow Capsule Classification

Workflow capsules SHALL be classified as governance artifacts.

They:

- SHALL NOT be directly executable
- SHALL require mediated ingestion
- SHALL require capability validation
- SHALL be subject to handshake scope rules

I.4.2 Handshake Binding of Workflow Context

If workflow capsules are exchanged under a handshake:

- They SHALL be cryptographically bound to the handshake identifier
- They SHALL inherit handshake lifetime constraints
- Revocation SHALL invalidate future activation
- Workflow semantics SHALL NOT be replayable across handshake boundaries

I.4.3 Replay and Idempotency

Replay evaluation SHALL consider:

- Idempotency mode
- Handshake state
- Node execution state
- PoAE™ logs (if available)

Replay SHALL NOT bypass:

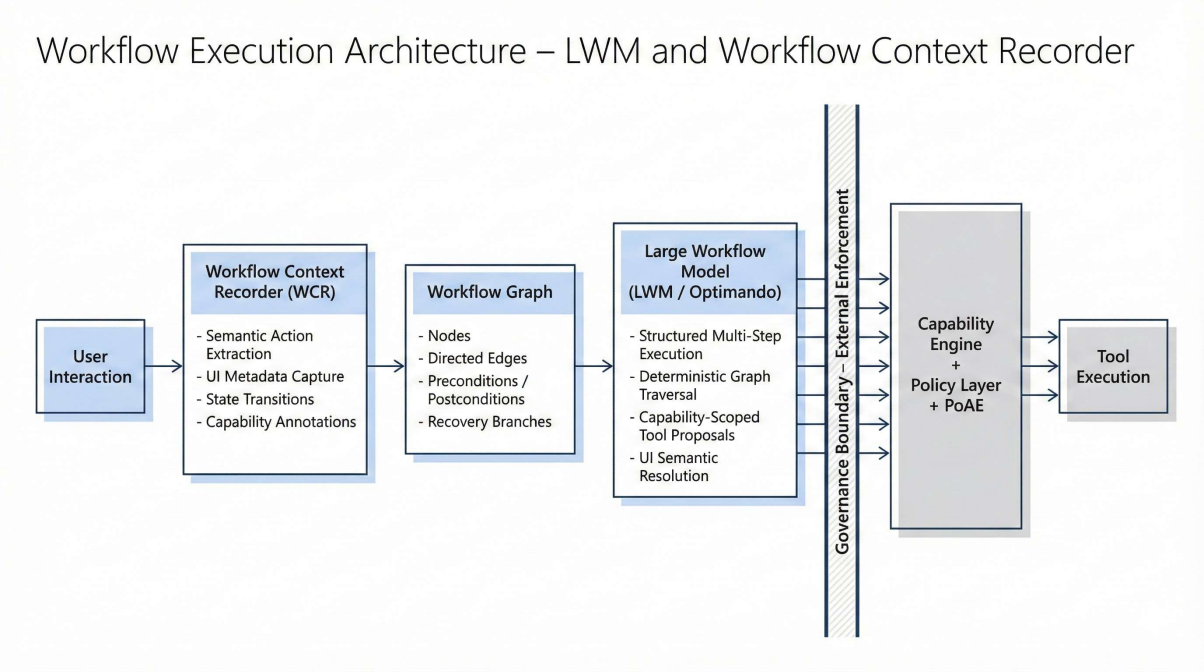
- Capability constraints
- Attestation requirements
- Purpose identifiers
- Consent conditions

I.4.4 Sensitive Workflow Context

Sensitive workflow elements:

- SHALL be stored exclusively within encrypted sub-capsules
- SHALL remain inactive until purpose-bound activation
- SHALL NOT influence deterministic execution logic prior to activation

---



I.5 Large Workflow Model (LWM) Enforcement

I.5.1 Definition

The Large Workflow Model (LWM / Optimando) is a workflow-specialized derivative of a foundation model optimized for:

- Structured multi-step task execution
- Deterministic workflow graph traversal
- Capability-scoped tool invocation
- UI abstraction and semantic resolution
- Recovery-aware reasoning

I.5.2 Authority Constraints

The LWM SHALL NOT constitute an autonomous authority layer.

Execution authority SHALL remain governed by:

- Capability engine
- Policy engine
- PoAE™ enforcement
- Attestation validation

I.6 Workflow Context Recorder (WCR)

I.6.1 Semantic Extraction Requirement

The Workflow Context Recorder SHALL:

- NOT record executable macros
- NOT produce replay scripts
- NOT bind execution to static selectors or pixel coordinates

It SHALL extract:

- Chronological semantic action sequences
- UI metadata (roles, labels, attributes)
- Structural DOM relationships
- Input-output dependencies
- State transitions
- Capability annotations

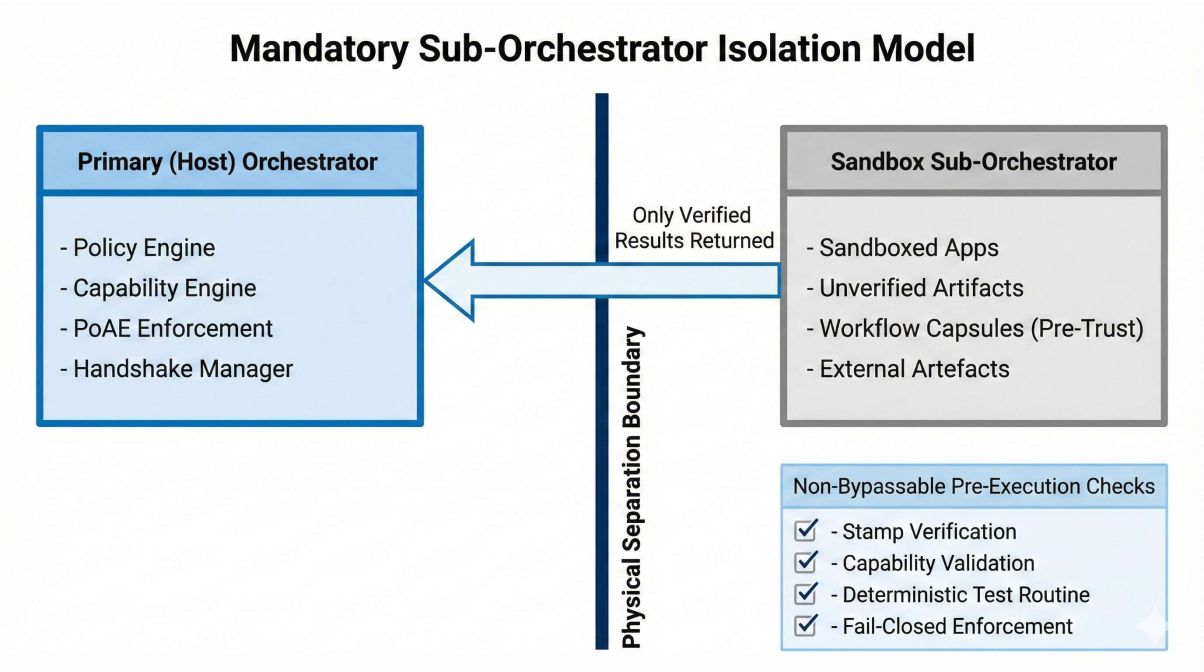
I.6.2 Structured Representation

Recorded workflows SHALL be transformed into workflow graphs containing:

- Semantic nodes
- Directed edges
- Capability annotations
- Optional recovery branches

Workflow capsules SHALL be structured textual artifacts.

They SHALL require mediated interpretation under BEAP™ governance.



I.7 Deterministic LLM-Governed App Ecosystem (Informative)

I.7.1 Sandboxed-by-Default Model

All applications and extensions SHALL begin with a mandatory sandbox label.
Trust escalation SHALL be explicit and never implicit.

I.7.2 Stamped Code Capsules

Artifacts SHALL be distributed as stamped code capsules binding:

- Executable code
- Dependencies
- Declared capabilities
- Identity
- Lineage metadata

Updates SHALL produce new stamped capsules with explicit parent references.

I.7.3 Network-Enforced Swarm Verification

Sandboxed artifacts SHALL undergo mandatory distributed deterministic testing.

- Test bundles SHALL be hash-validated and whitelisted
- Trust progression SHALL follow fail-closed semantics

Marketplace eligibility SHALL require:

- Sufficient valid test artifacts
- Coverage across bundle families
- Participation from higher-trust identities

I.7.4 Deterministic LLM Evaluation

Formal trust decisions SHALL rely on deterministic evaluation:

- Pinned model versions
- Deterministic decoding
- Schema-validated outputs
- Reproducible pipelines

Evaluations SHALL produce hard pass/fail gates.

I.8 High-Assurance Mode

In High-Assurance Mode:

- All executable components SHALL be WRStamped
- All versions SHALL be pinned
- All execution SHALL be identity-bound
- Dynamic uncontrolled runtime code SHALL be excluded

Unverifiable content SHALL be routed off-band.

I.9 Security and Determinism Invariants

The enforcement model SHALL ensure:

- Processing restricted to declared purposes
- No implicit data access
- Full auditability
- Attestation-bound activation
- Strict determinism

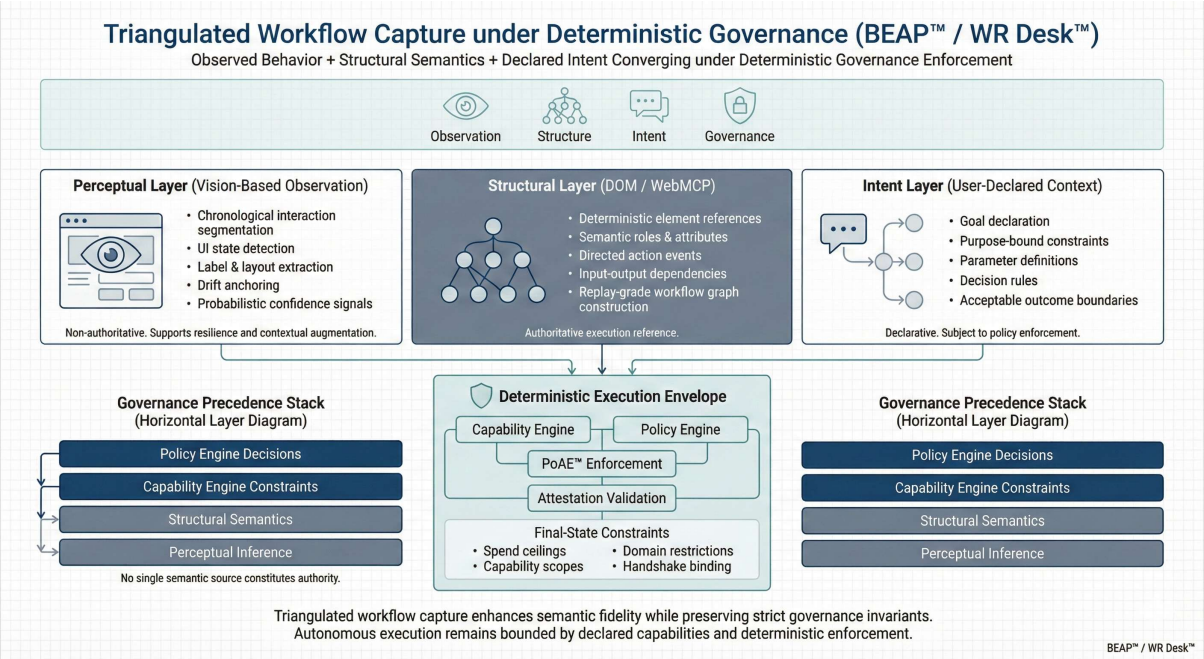
Handshake revocation SHALL immediately affect future activation without altering historical integrity.

I.10 Staged Evolution

This Annex defines the target enforcement architecture.

Partial implementations do not reinterpret this specification.

Conforming systems SHALL evolve toward full compliance without diluting guarantees.



I.11 Triangulated Workflow Capture Model (Informative)

I.11.1 Purpose and Rationale

This section describes the tri-layer workflow capture model employed by WR Desk™ for high-fidelity semantic workflow reconstruction.

It clarifies how observational signals, deterministic structural metadata, and user-declared intent are combined to produce governance-bound workflow capsules.

This section is informative and does not modify normative requirements defined elsewhere in this Annex or in the Canon.

I.11.2 Three-Layer Capture Architecture

Workflow capture MAY integrate three independent but synchronized semantic layers:

1. **Perceptual Layer (Vision-Based Observation)**
2. **Structural Layer (DOM / WebMCP / Deterministic Interface Metadata)**
3. **Intent Layer (User-Declared Semantic Context)**

These layers serve distinct roles and SHALL NOT be conflated.

I.11.3 Perceptual Layer (Vision-Based Observation)

The perceptual layer:(Vision AI, Smartglass, Camera)

- Observes rendered UI states
- Segments chronological interaction sequences
- Detects state transitions
- Extracts visible labels and layout context
- Provides recovery anchoring in case of structural drift

The perceptual layer:

- Is probabilistic in nature
- SHALL NOT constitute execution authority
- SHALL NOT override structural validation
- MAY provide confidence signals for reconciliation

Its primary role is resilience and contextual augmentation.

I.11.4 Structural Layer (Deterministic Interface Semantics)

The structural layer:

- Extracts DOM relationships or equivalent interface metadata
- Resolves semantic roles, identifiers, and attributes
- Records directed action events
- Captures input-output dependencies
- Establishes deterministic execution references

The structural layer:

- Constitutes the authoritative execution reference
- Enables reproducible workflow graph construction
- Supports deterministic replay under governance
- Anchors capability validation

When available, structural signals SHALL take precedence over perceptual hypotheses for execution semantics.

I.11.5 Intent Layer (User-Declared Semantic Context)

The intent layer:

- Captures user-declared goals
- Captures purpose-bound constraints
- Defines parameterization
- Defines decision rules
- Defines acceptable outcomes

The intent layer:

- SHALL NOT directly execute logic
- SHALL NOT expand capability scopes

- SHALL remain subject to PoAE™ enforcement
- SHALL bind workflows to declared purposes

Intent declarations augment semantic meaning but do not override deterministic enforcement layers.

I.11.6 Synchronization and Alignment

All three layers MAY be time-aligned via chronological indexing.

During workflow graph construction:

- Structural data defines executable nodes
- Intent data defines semantic purpose bindings
- Perceptual data assists drift handling and anomaly detection

Conflicts SHALL be resolved according to governance precedence:

1. Policy engine decisions
2. Capability engine constraints
3. Structural layer semantics
4. Perceptual inferences

Intent descriptions SHALL remain declarative and non-authoritative.

I.11.7 Deterministic Envelope for Autonomous Execution

When combined with:

- Capability enforcement
- Policy validation
- PoAE™ constraints
- Attestation-bound execution contexts

The tri-layer capture model enables a deterministic execution envelope.

Autonomous execution SHALL remain bounded by:

- Explicit capability declarations
- Final-state constraints
- Spend or action ceilings
- Domain or scope restrictions
- Handshake bindings (if applicable)

No workflow SHALL exceed declared and validated enforcement constraints, regardless of inferred intent.

I.11.8 Non-Executable Recording Principle

The Workflow Context Recorder (WCR):

- Does not produce executable macros
- Does not bind to pixel coordinates
- Does not generate uncontrolled replay scripts

Instead, it produces governance-bound workflow graphs requiring mediated interpretation under BEAP™.

I.11.9 Assurance Implications

In High-Assurance Mode:

- Tri-layer capture MAY be used to increase semantic fidelity
- Deterministic structural validation remains mandatory
- Perceptual inference SHALL NOT relax enforcement constraints
- All resulting workflow capsules remain subject to handshake binding and attestation requirements

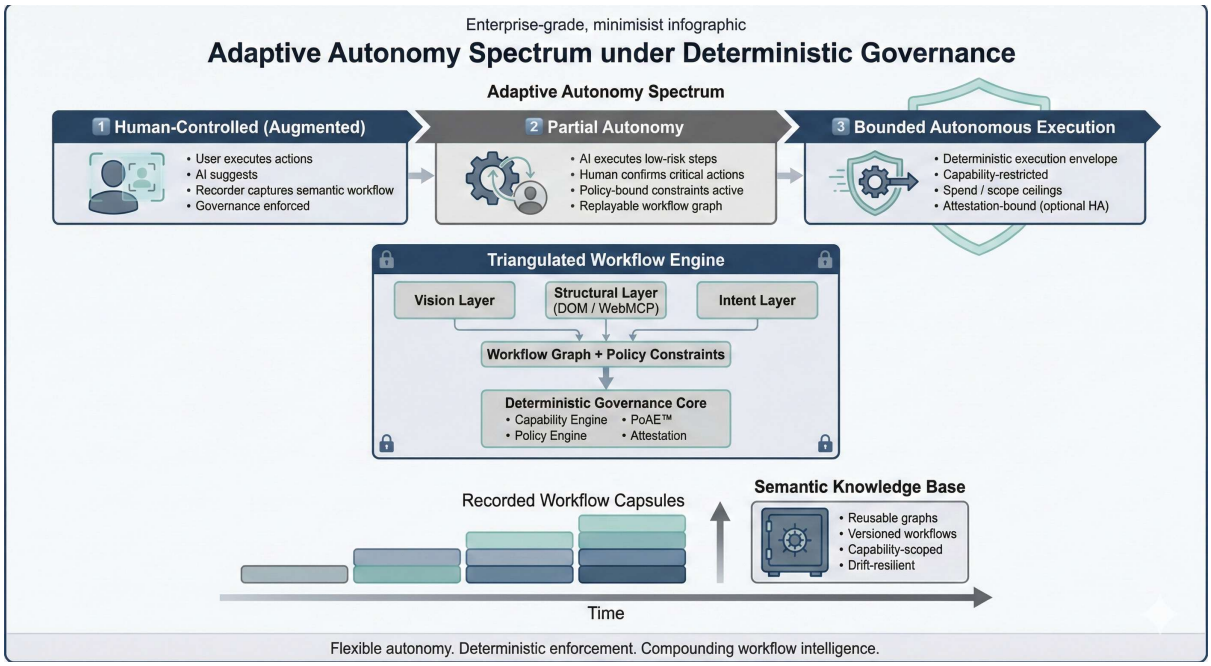
Tri-layer capture enhances robustness but does not modify governance invariants.

I.11.10 Design Principle

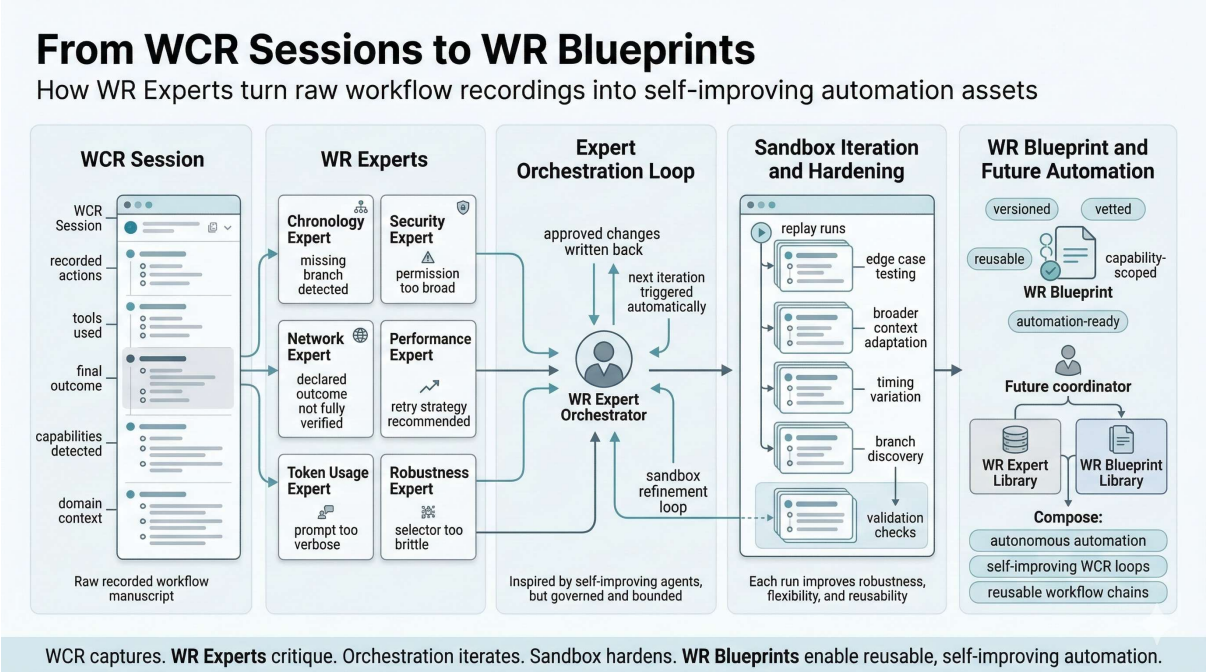
The tri-layer model embodies the following principle:

Observed behavior, structural reference, and declared intent SHALL converge under deterministic governance enforcement.

No single semantic source SHALL constitute authority.



The long-term vision is a flexible autonomy spectrum built on a constant governance core. Users could operate in fully human-controlled (augmented), partially autonomous, or strictly bounded autonomous modes—without changing the underlying enforcement architecture. Over time, recorded workflows would evolve into structured, policy-bound knowledge assets, forming a reusable and drift-resilient execution layer. Autonomy would scale progressively, but always remain constrained by declared capabilities, explicit purpose, and verifiable governance controls.



From WCR Sessions to WR Blueprints: Expert-Orchestrated Workflow Hardening

A Workflow Context Recorder should do more than capture what happened during a session. Its real value begins when a recorded workflow can be refined into a reusable, trustworthy artifact. That is where **WR Blueprints** and **WR Experts** become important.

A **WR Blueprint** is the vetted form of a recorded workflow. It is not just a log of clicks, inputs, and outputs, but a structured and versioned representation of how a task was completed, which capabilities were required, what outcome was achieved, and which constraints, risks, and edge cases were identified. A raw WCR session becomes operationally useful only once it has been hardened into a WR Blueprint.

For routing and reuse, title and description alone are not enough. A stronger model should also consider the tools actually used, the final outcome reached, the capability profile, the domain context, and the observed execution pattern. These signals are more reliable for machine routing than human-written labels alone and make it easier to find relevant workflows later.

This is where **WR Experts** come in. WR Experts are specialist evaluators that review a WCR session from different angles and provide concise, structured feedback. One expert may focus on chronological accuracy and whether the sequence of actions is complete. Another may analyze network behavior to verify that the workflow behaves as declared. A security expert may identify risks, overbroad permissions, or unsafe side effects. Other experts may focus on performance, token usage, robustness, recoverability, capability minimization, or broader execution quality.

The key idea is that WR Experts do not have to operate in isolation. They can themselves be **orchestrated**. Their findings can be merged by a coordinator that writes approved changes directly back into the **WCR manuscript** and automatically triggers the next iteration loop. This creates a bounded form of recursive improvement inspired by self-improving agent systems: one expert identifies a weakness, another proposes a stabilization, the coordinator

incorporates the change, and the workflow is replayed again under sandbox conditions. The result is not uncontrolled self-modification, but structured iterative hardening.

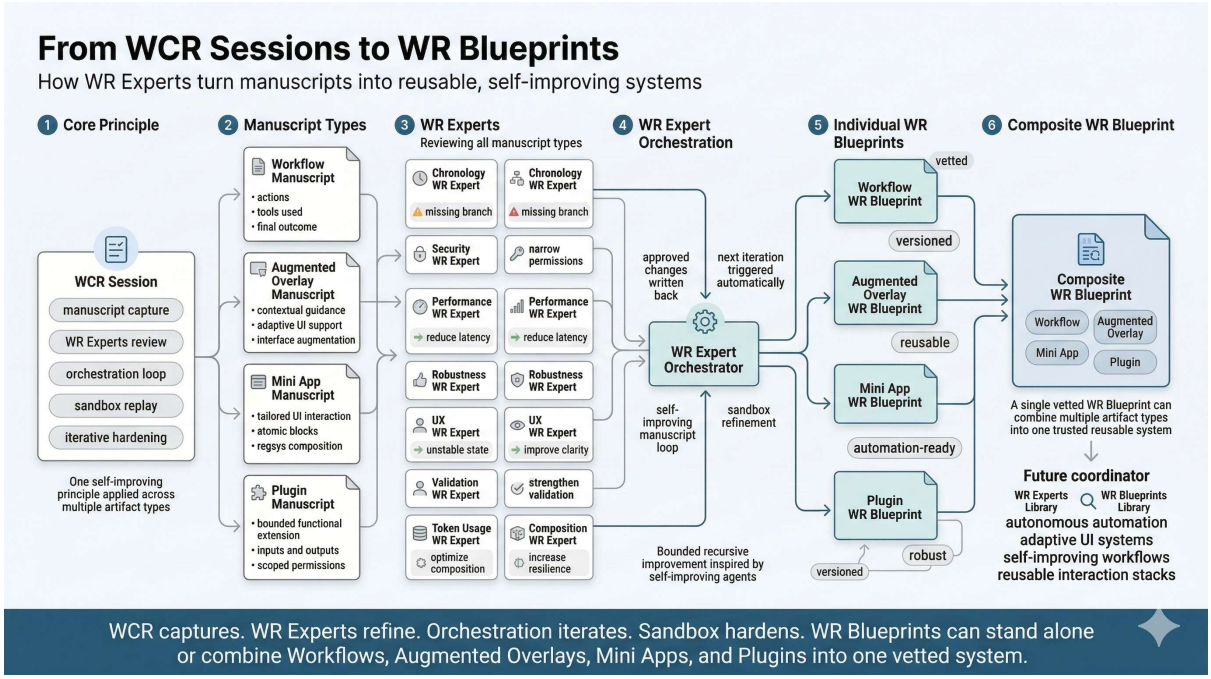
This makes the recorder more than a passive capture tool. During recording, WR Experts can surface live feedback in a grid or panel so the human operator sees quality signals while the workflow is still being authored. The system can point out missing branches, brittle selectors, weak outcome definitions, redundant steps, or unnecessary permissions. Low-risk improvements can be applied immediately to the sandbox version, while higher-impact changes remain suggestions until validated.

After recording, the workflow should be run several times in a sandbox. Each replay can test edge cases, timing variations, partial failures, or broader use contexts. The purpose is to make the workflow more flexible, robust, and reusable. With every iteration, the WCR manuscript can be refined, weaknesses can be reduced, and the workflow can move closer to the quality expected of a WR Blueprint.

A coordinator layer becomes especially powerful once a larger ecosystem exists. Over time, the coordinator should be able to search the **WR Expert library** and the **WR Blueprint library** to assemble highly autonomous, self-improving automation loops. In that model, existing WR Experts can be reused as evaluators, while proven WR Blueprints can be chained together as building blocks for more advanced workflows. The system can then not only record and improve workflows, but also compose and refine new ones using previously validated assets.

That creates a strong progression model. WCR captures the session. WR Experts analyze and improve it. The coordinator orchestrates iterations and feeds accepted changes back into the manuscript. Sandbox replays test for stability and edge cases. Once the workflow has matured and passed the required checks, it is promoted to a WR Blueprint.

In this form, WR Blueprints and WR Experts are not just naming additions. Together they define a scalable path from raw workflow capture to expert-guided, self-improving, reusable automation.



From Manuscripts to WR Blueprints: Self-Improving Workflows, Augmented Overlays, Mini Apps, and Plugins

The same principles used in a self-improving **WCR session** can be applied not only to workflows, but also to **Augmented Overlays**, **Mini Apps**, and even **plugins**. The WCR model should therefore be understood as a general method for creating, refining, and hardening reusable operational artifacts. In all of these cases, the process begins with a manuscript and matures through iteration into a vetted **WR Blueprint**.

A workflow may begin as a captured sequence of actions and outcomes. An Augmented Overlay may begin as a manuscript describing contextual interface guidance. A Mini App may begin as a structured description of a tailored UI interaction assembled from trusted atomic blocks. A plugin may begin as a bounded functional extension with clearly defined inputs, outputs, permissions, and responsibilities. In every case, the first version is only a draft. It defines intent and structure, but it is not yet a hardened reusable asset.

Just like in a WCR session, **WR Experts** can evaluate these manuscripts from multiple perspectives while they are being created and refined. Depending on the artifact type, WR Experts may assess chronology, security, robustness, logic flow, state handling, UX quality, token usage, performance, composition quality, validation coverage, or contextual relevance. Their role is to provide concise, structured feedback that improves the manuscript without weakening control or clarity.

These **WR Experts** can also be orchestrated. Their findings can be merged by a coordinator that writes approved changes directly back into the manuscript and automatically triggers the next iteration loop. This creates a bounded self-improvement process inspired by autonomous agents: weaknesses are detected, refinements are proposed, low-risk changes are applied, and the result is tested again. In that way, the manuscript improves continuously during and after authoring.

The same replay and hardening model from WCR sessions applies across all artifact types. A workflow can be replayed across edge cases and broader task contexts. An Augmented Overlay can be tested across different layouts, interface states, and user situations. A Mini App can be exercised across different inputs, flows, and UI conditions. A plugin can be validated across dependency, permission, and execution scenarios. Each accepted improvement creates a stronger next version.

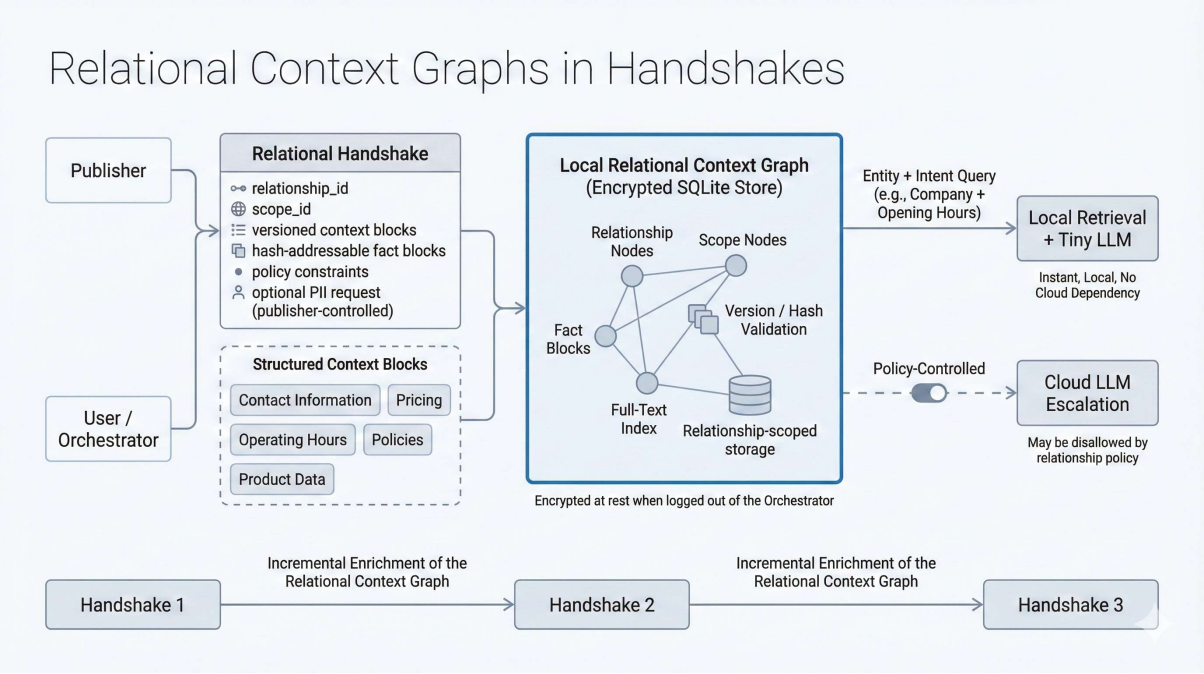
This is where **WR Blueprints** become central. The same principles of the WCR session can be applied to create **WR Blueprints** for workflows, Augmented Overlays, Mini Apps, and plugins. A raw manuscript becomes a candidate, **WR Experts** refine it, sandbox iterations harden it, and once it is sufficiently mature, it is promoted into a vetted **WR Blueprint**.

A particularly important extension is that a **WR Blueprint** does not have to remain limited to a single artifact type. A later-stage **WR Blueprint** can also combine **workflows, Augmented Overlays, Mini Apps, and plugins into one single vetted WR Blueprint**. This means the final artifact is not just an isolated reusable component, but a higher-order operational package. A workflow can include embedded Mini Apps for tailored interaction, Augmented Overlays for contextual guidance, and plugins for bounded functional extension, all governed and validated together as one vetted unit.

This creates a much more powerful model for reuse. Instead of storing only separate assets, the system can also maintain composite **WR Blueprints** that unify multiple layers of behavior and interaction. Over time, libraries of **WR Blueprints** and **WR Experts** can

support increasingly advanced compositions, where proven components are not only reused individually but assembled into larger self-improving systems.

The lifecycle remains consistent across all domains: capture or author a manuscript, let **WR Experts** evaluate it, orchestrate improvements, replay and harden it in sandbox, and promote the mature result into a **WR Blueprint**. From there, vetted **WR Blueprints** can also be combined into more comprehensive **WR Blueprints** that integrate workflows, Augmented Overlays, Mini Apps, and plugins in a single reusable and trusted artifact. That is what turns the WCR approach from a recording method into a scalable architecture for self-improving systems.



Local Persistence and Incremental Enrichment – Relational Context Graph

As part of the relational handshake, exchanged context is not treated as transient prompt material but as durable relationship state. Each handshake may carry one or more semantically defined *context blocks* (fact blocks and/or structured semantic text) that are explicitly bound to a `relationship_id` and optionally to a `scope_id` (e.g., product, location, contract, session). These blocks are versioned and hash-addressable, enabling deterministic identification, deduplication, and update detection. A block is typically *created once* (initial publication), and subsequent handshakes or messages may either reference the existing block by `(block_id, hash)` or enrich the relationship by attaching incremental relational context data (relationship-specific facts) and/or general context data (publisher-wide facts). This yields an append/enrich model rather than repeated re-transmission of the full context payload.

Local Encrypted Storage Model

Upon receipt, the orchestrator persists the handshake-derived context blocks into a local SQLite knowledge store to make them immediately searchable and LLM-native at low latency. Storage is organized around relationship scoping (e.g., indexing by

`relationship_id`, `publisher_id`, `topic/type`, `valid_until`) to enable fast entity+intent retrieval across the accumulated archive of BEAP capsules and handshake context. The SQLite store is encrypted at rest when the orchestrator is logged out: on logout, encryption is enforced by closing the database, removing keys from memory, and rendering the store unreadable until the next authenticated login session. When logged in, the orchestrator can query the local store directly, enabling instant retrieval and local processing without cloud dependency.

Policy-Governed Escalation Path

For increased reasoning capacity or broader context windows, the orchestrator may provide a user-controlled escalation toggle to route selected requests to cloud-based LLMs. This escalation is not implicit: it is explicitly gated by the active relationship policy negotiated in the handshake (and by any applicable local policy). If escalation is disallowed by policy, the orchestrator must remain local-only and restrict processing to locally available context and models. If allowed, the orchestrator can selectively transmit only the minimal required snippets derived from the local context graph, preserving the relationship-scoped constraints while enabling higher-tier inference when needed.

Terminology

WRVault™ (Normative)

WRVault™ denotes a locally controlled, encrypted, multi-layered secure vault bound to executing identity and environment.

Data SHALL be:

- Compartmentalized
- Scope-restricted
- Purpose-restricted
- Cryptographically isolated by class
- Exposed only through controlled augmented contexts

License Notice

This Annex forms an integral part of the WR Desk™ specification and is licensed under the same terms as BEAP™, WR Desk™, and PoAE™.